

Contents

CUAD Legal-Clause Pipeline — Training & Testing Report	1
What this is	1
Train / Test Split	2
Training Results	2
Stage 2A — Presence classifier	2
Stage 2B — Span extractor	3
Stage 1 — Summarizer	3
Test Results (20% held-out — 102 contracts)	3
Stage 2A — Presence classifier: Confusion Matrix	3
Stage 2B — Span extractor test results	4
Stage 1 — Summarizer test results	5
Does the AND rule punish rare categories? (<code>analysis/rare_tail.py</code>)	5
End-to-end: what actually reaches the user (<code>analysis/e2e.py</code>)	6
Sparse attention: the cost, measured (<code>analysis/lf_one.py</code>)	6
Risk taxonomy — validation status	7
Risk-weighted evaluation (<code>analysis/risk_threshold_calib.py</code>)	7
Threshold and calibration	7
Aggregation ablation (<code>analysis/pooling_ablation.py</code>)	8
Dataset-split variance (<code>analysis/split_variance.py</code>)	8
Summary scorecard	8
Key design decisions	9
Engineering notes	9
Files	9

CUAD Legal-Clause Pipeline — Training & Testing Report

AI-Powered Legal Document Analysis (CUAD) · BRAC University Full-dataset build, EDA → three models, 80/20 train-test split with confusion matrix evaluation. Updated 2026-08-22 (second-review revision: risk-weighted evaluation, threshold/calibration, aggregation ablation, split variance; earlier supervisor revision: training-budget control, seed variance, span and end-to-end evaluation, summarizer reference-set pilot, sparse-attention benchmark).

What this is

A from-scratch build of the three-model CUAD pipeline on the **full dataset** (510 contracts), done interactively in a Jupyter notebook on an Apple-Silicon laptop. The dataset is split **80/20 at the contract level** — 408 contracts for training, 102 contracts for testing — ensuring no clause from a training contract leaks into the test set.

The three models:

Stage	Model	Job
2A	Presence classifier	For each of 41 clause categories, is it present in this contract?
2B	Span extractor	Locate the exact clause text (SQuAD-style QA).
1	Summarizer	Rewrite a clause as one plain-English sentence.

Train / Test Split

	Contracts	(contract x category) rows	Positive rate
Train (80%)	408	16,728	32.0%
Test (20%)	102	4,182	32.2%
Total	510	20,910	32.1%

The split is **contract-level** (whole contracts assigned to one side), seeded (**SEED=42**), and reused across all three models so “test” means the same 102 unseen contracts everywhere.

Split files: `data/train_80.csv` and `data/test_20.csv`.

Training Results

Stage 2A — Presence classifier

TF-IDF + Logistic Regression baseline (trained on 80%)

Metric	Value
micro-F1	0.775
weighted-F1	0.777
macro-F1 (all 41 categories)	0.572
macro-F1 (≥ 10 test positives, 34 cats)	0.666

DistilBERT window-level classifier (MIL) - 53,662 window examples (39.2% positive), subsampled to 24,000 for training - 2 epochs, batch 16, LR 2e-5, max length 256 - Training loss: 0.319 $\rightarrow$ 0.261; validation loss: 0.278 $\rightarrow$ 0.257 - Training time: ~52 min on Apple MPS - Standalone micro-F1 on test: **0.694**

AND-ensemble (TF-IDF + transformer, both must agree) - micro-F1: **0.776** — nominally the best model, but see the significance test below: its margin over the TF-IDF baseline is not statistically distinguishable from zero.

Control 1 — was the transformer just undertrained? (`analysis/presence_run.py`) The baseline sees all 408 contracts in full; the transformer saw 24,000 of 53,662 windows. We retrained on the **full 53,662 windows** (140.7 min), everything else identical:

Transformer training windows	micro-F1	Precision	Recall
24,000 (44.7%)	0.6943	0.5620	0.9080
53,662 (100%)	0.6939	0.5546	0.9266

Difference -0.0004, 95% CI [-0.0098, +0.0095] — a tie. The gap to TF-IDF holds at **+0.0809** (CI [+0.0649, +0.0980]). More data made the model fire *more* (recall up, precision down), which is what max-pooling over ~35 windows predicts. **The deficit is architectural, not a training-budget artifact.**

Control 2 — how much moves if we only change the seed? Three runs at the 24,000 budget:

Model	Seed 42	Seed 43	Seed 44	Mean	SD	Range
Transformer	0.6943	0.6928	0.7082	0.6984	0.0085	0.0154
AND ensemble	0.7761	0.7774	0.7803	0.7780	0.0022	0.0042

The ensemble’s SD (0.0022) and range (0.0042) both **exceed its +0.0014 margin** over the baseline — independent confirmation of the tie. The baseline’s lead over the transformer is 9.5 SDs, so that finding is seed-robust. Note the ensemble is 4x *less* seed-variable than the transformer alone.

Stage 2B — Span extractor

- DistilBERT-QA, 8,000 training examples (from 11,156 total train)
- 2 epochs, batch 16, LR 3e-5, max length 320
- Training time: ~23 min on Apple MPS

Stage 1 — Summarizer

- FLAN-T5-small, 4,000 training examples (from 11,156 total train)
- 2 epochs, batch 8, LR 3e-4
- Training loss: 0.141 $\rightarrow$ 0.085; validation loss: 0.096 $\rightarrow$ 0.075
- Training time: ~8 min on CPU (`use_cpu=True` to avoid MPS OOM)

Test Results (20% held-out — 102 contracts)

Stage 2A — Presence classifier: Confusion Matrix

AND-Ensemble (TF-IDF + DistilBERT), evaluated on 4,182 (contract x category) test predictions:

	Predicted Absent	Predicted Present
Actually Absent	TN = 2,523	FP = 311
Actually Present	FN = 296	TP = 1,052

Metrics derived from the confusion matrix:

Metric	Value
Accuracy	85.49%
Precision	77.18%
Recall (Sensitivity)	78.04%
Specificity	89.03%
F1 Score	0.7761

Per-category F1 (top 10 and bottom 5):

Category	Test positives	Precision	Recall	F1
Document Name	102	1.000	1.000	1.000
Parties	101	0.990	1.000	0.995
Governing Law	90	0.956	0.967	0.961
Agreement Date	93	0.929	0.989	0.958
Expiration Date	84	0.914	0.881	0.897
License Grant	56	0.907	0.875	0.891
Anti-Assignment	74	0.831	0.932	0.879
Effective Date	73	0.831	0.877	0.853
Cap On Liability	62	0.879	0.823	0.850
Insurance	34	0.725	0.853	0.784

Category	Test positives	Precision	Recall	F1
...				
Most Favored Nation	7	0.000	0.000	0.000
No-Solicit Of Customers	6	0.000	0.000	0.000
Price Restrictions	4	0.000	0.000	0.000
Source Code Escrow	2	0.000	0.000	0.000
Unlimited/All-You-Can-Eat-License	2	0.000	0.000	0.000

Categories with 0.000 F1 have ≤ 7 test positives — too few to learn reliably with 510 total contracts.

Ensemble comparison (all on the same 20% test set):

Model	micro-F1
Ensemble AND (both must agree)	0.776
TF-IDF baseline	0.775
Ensemble AVG	0.718
Transformer (max-pool) alone	0.694
Ensemble OR	0.696

Significance — which of these gaps are real? A cluster bootstrap over the 102 test contracts (resampling contracts, not the correlated within-contract cells; 10,000 resamples, `analysis/bootstrap_ci.py`):

Comparison	Difference	95% CI	P(>0)	Verdict
AND-ensemble vs TF-IDF baseline	+0.0014	[-0.0068, +0.0089]	0.626	tie
TF-IDF baseline vs transformer	+0.0805	[+0.0626, +0.0992]	1.000	real
AND-ensemble vs transformer	+0.0818	[+0.0654, +0.0987]	1.000	real

So the headline should be read carefully: the ensemble and the baseline are **tied** on this test set, while the windowed transformer’s deficit against both is large and significant. The ensemble is still the model we deploy, but for its balanced error profile (precision 77.18% / recall 78.04%) rather than for its rank.

Stage 2B — Span extractor test results

Evaluated on **2,667 test examples** (answer-bearing windows from the 102 test contracts):

Metric	Score	95% CI (by contract)	95% CI (by clause)
token-F1	0.763	[0.740, 0.782]	[0.751, 0.774]
Exact Match	35.5%	[32.5%, 38.6%]	[33.7%, 37.3%]
Overlap (F1 ≥ 0.5)	82.7%	[79.7%, 85.1%]	[81.3%, 84.1%]

The same cluster bootstrap used for presence, applied here (`analysis/span_ci.py`). The contract-clustered interval is $\sim 2\times$ wider than treating clauses as independent — that factor is the cost of ignoring the correlation between clauses in the same contract. Differences below ~ 2 points of token-F1 are not measurable on this test set.

Scope: these numbers assume the model is handed an answer-bearing window. See the end-to-end section below for what happens when the window is chosen by Stage 2A.

Stage 1 — Summarizer test results

Evaluated on **150 test clauses** (sampled from the 102 test contracts):

The 41 “gold” targets were **written by us**, not annotated — every one of the 2,667 test clauses is scored against one of only 41 unique sentences. To separate template reproduction from summarization we scored the *same* 150 clauses against a second reference set: 150 clause-specific summaries drafted from the clause text alone, independently of the model (`analysis/diverse_refs.json`, `analysis/summ_pilot.py`).

System	Reference set	ROUGE-L	95% CI
Fine-tuned FLAN-T5	41 category templates	0.775	[0.718, 0.833]
Fine-tuned FLAN-T5	clause-specific (pilot)	0.116	[0.105, 0.128]
Zero-shot FLAN-T5	clause-specific (pilot)	0.299	[0.275, 0.322]
<i>Template alone, no model</i>	clause-specific (pilot)	0.115	[0.104, 0.127]

The fine-tuned model emits one of the 41 templates verbatim on 100% of test clauses — the correct one 70% of the time, a *different category’s* template the other 30%. The bare template with no model scores 0.115, statistically identical to the model’s 0.116: the model adds nothing measurable over a lookup table. Zero-shot scores higher (0.299) because it copies source wording, which inflates a longest-common-subsequence metric — it is not a better summarizer, but fine-tuning did trade away whatever clause-specific content the base model had.

Conclusion: this is a working **classification-to-template** system, useful in the app, and it should not be described as free-text summarization. The number that describes clause-level summarization ability is 0.116.

Caveat on the pilot references: they were drafted with an LLM assistant, not written by a lawyer or independently by a human annotator. They are independent of the model, which is what makes the comparison valid, but they are a pilot instrument rather than gold annotation.

Does the AND rule punish rare categories? (`analysis/rare_tail.py`)

Requiring two models to agree is a stricter bar exactly where each has least training signal. Splitting the 41 categories by **training** frequency, bottom 10 vs the rest:

Group	Model	Precision	Recall	F1	TP	FN
Rarest 10 (70 test positives)	Transformer	0.261	0.500	0.343	35	35
	TF-IDF	0.429	0.300	0.353	21	49
	AND ensemble	0.586	0.243	0.343	17	53
Other 31 (1,278 test positives)	Transformer	0.582	0.930	0.716	1,189	89
	TF-IDF	0.753	0.838	0.793	1,071	207
	AND ensemble	0.776	0.810	0.792	1,035	243

On common categories the ensemble works as advertised (+0.077 F1 over the transformer). **On the rare tail it nets exactly zero** — it buys 32.5 points of precision with 25.7 points of recall, cutting detections from 35/70 to 17/70. On five of the ten rarest categories it scores 0.000, and on three of those the transformer alone was finding something before the conjunction zeroed it out.

Deployment implication: *Most Favored Nation* is in this tail and is a **High-risk** category — both models score 0.000 on it. A signer is arguably harmed more by a missed rare high-risk clause than by a missed common one, so a deployment aimed at protecting the user would run a permissive rule on High-risk categories and the conjunction elsewhere. Not implemented — choosing it after seeing the test set would invalidate it.

End-to-end: what actually reaches the user (analysis/e2e.py)

Every number above is measured with the other stages held out of the way. Run in series on all 102 test contracts, asking of each of the 1,348 gold clauses whether it survives **all three** stages:

Stage	Condition	Surviving	Share
—	gold clauses in test set	1,348	100.0%
2A Presence	category detected	1,052	78.0%
2B Span	extracted text overlaps gold at F1 ≥ 0.5	293	21.7%
Risk	correct level attached	293	21.7%
End-to-end		293	21.7% (95% CI [19.6%, 23.9%])

Multiplying the component scores (78.0% x 82.7%) predicts ~64%. The pipeline delivers a third of that. Where the clauses are lost, over the 1,052 whose category was detected:

	Count	Share
Selected window contains the gold clause	718	68.3%
— and the span model then found it	285	39.7% of 718
— and the span model missed it	433	60.3% of 718
Selected window does not contain the clause	334	31.7%

Two distinct causes: 1. **The max-pooled window is often the wrong window** (31.7%). Max-pooling was trained to answer “does this window contain such a clause”, not “which window is it in” — it is being reused as a locator. 2. **The span model is weak on windows it was not trained to see.** Even with the right window it succeeds 39.7% of the time vs 82.7% isolated, because training re-centred every focus window so the answer began ~150 chars in. Legitimate (no leakage — it never touches an inference input) but it narrowed the distribution the model can read. Mean token-F1 falls 0.763 $\rightarrow$ 0.386.

Both have cheap fixes that need no extra parameters: train the span model on varied answer offsets, and choose the extraction window with a locator trained for locating.

For the app: the risk badge and category name are far more reliable than the quoted sentence beneath them. A clause card should be read as “look here” rather than “this is the clause”.

Sparse attention: the cost, measured (analysis/lf_one.py)

The design rationale rejected Longformer partly on cost. Measured on the M4 (16 GB unified memory), Longformer-base at its published geometry (148.7M params, 4,096 tokens):

Model	Operation	Batch	Time	Peak memory
Longformer-base	forward, 4,096 tok	1	3.69 s	2.18 GB
DistilBERT	forward, 256 tok	16	0.19 s	1.15 GB
Longformer-base	training step	1	258 s	14.71 GB
Longformer-base	training step	2	2,170 s	20.75 GB
DistilBERT	training step, 256 tok	16	3.17 s	4.47 GB

This corrects us. We had claimed fine-tuning a Longformer was “far beyond a single laptop’s memory budget”. It is not — at batch size 1 it fits, in 14.71 GB of 16 GB. The memory claim only holds from batch

2 (20.75 GB, past physical memory, and 8.4x slower per step from paging). The real barrier is time: at 258 s/step our two-epoch schedule would take ~136 days versus 47 minutes for DistilBERT. Inference: scanning one median contract for all 41 categories takes ~10.6 s with DistilBERT vs ~454 s (7.5 min) with Longformer — which rules it out for an interactive app regardless of training hardware.

Risk taxonomy — validation status

The 8/22/11 High/Medium/Low taxonomy over all 41 categories was authored **entirely by the three of us**; none of us has legal training. It is the main value-add over CUAD and it drives what the app tells a user to worry about, so its evidential basis matters. **As of this writing no legal professional has reviewed it, and no validation number is reported anywhere.** A review instrument is ready at `review/risk_taxonomy_review_sheet.csv` (41 rows, ~30 min for a reviewer). Until it is completed, the taxonomy is a documented, auditable, explicitly **unvalidated** design artifact.

Risk-weighted evaluation (`analysis/risk_threshold_calib.py`)

Micro-F1 treats all 41 categories equally; a signer does not. Splitting by the taxonomy’s own bands:

Band	Model	Precision	Recall	F1	Found	Missed
High (8 cats, 176 pos.)	Transformer	0.404	0.841	0.546	148	28
	TF-IDF	0.540	0.648	0.589	114	62
	AND ensemble	0.585	0.625	0.604	110	66
	OR ensemble	0.391	0.864	0.538	152	24
Medium (22 cats, 498 pos.)	AND ensemble	0.647	0.673	0.659	335	163
Low (11 cats, 674 pos.)	AND ensemble	0.924	0.901	0.912	607	67

The headline configuration is the worst one for High-risk clauses. AND misses **37.5%** of them; the transformer alone misses 15.9%; OR — which finishes *last* on aggregate micro-F1 — misses 13.6%. Choosing AND on micro-F1 costs a signer 42 of 176 High-risk clauses. Two thirds of the aggregate’s mass is Low-risk (dates, party names, governing law), so **micro-F1 and user harm point in opposite directions**. We report this as a diagnosis, not a retuning: switching to OR after seeing the test set would be test-set selection.

Threshold and calibration

The 0.5 threshold was never chosen — it is the softmax default. Sweeping it post hoc:

Model	F1 @ 0.5	Best F1	at threshold	High-risk recall at best
Transformer (max-pool)	0.6943	0.7504	0.90	0.688 (down from 0.841)
AND ensemble	0.7761	0.7761	0.50	0.625

So a meaningful part of the transformer’s reported deficit is an un-chosen operating point rather than an inability to discriminate. We do not adopt 0.9 (that would be test-set tuning), but the qualification belongs on the finding.

Calibration. The score the app displays as “confidence” has **ECE 0.201**, Brier 0.194, and is overconfident throughout: scores above 0.9 correspond to the clause being present ~73% of the time; scores in [0.5, 0.6) to ~20%. Max-pooling is not a probability-preserving operation. Platt scaling on a validation split would fix this without retraining.

Aggregation ablation (`analysis/pooling_ablation.py`)

Max-pooling was chosen because it *is* the MIL rule, never compared against alternatives. Scoring every window once and varying only the rule:

Aggregation	F1 @ 0.5	Best F1	at threshold
Max (thesis)	0.6943	0.7504	0.90
Top-3 mean	0.7144	0.7210	0.40
Top-5 mean	0.6661	0.7192	0.30
Mean (all windows)	0.1664	0.7114	0.10
Noisy-OR	0.5897	0.7022	0.95
Second-highest	0.6832	0.7016	0.30
<i>TF-IDF baseline</i>	0.7747	–	–

At the threshold actually used, top-3 mean beats max. Given its own threshold, max is best of the six. **But no rule at any threshold reaches the baseline** – which closes off the last explanation for the transformer’s defeat: not budget, not seed, not aggregation, not threshold.

Dataset-split variance (`analysis/split_variance.py`)

Model-seed variance holds the test set fixed, so it cannot see split effects. Refitting the TF-IDF baseline on 20 independent contract-level splits:

	Mean	SD	Min	Max	Range
micro-F1 over 20 splits	0.7829	0.0063	0.7702	0.7918	0.0216

Split SD (0.0063) is the largest of the three uncertainties measured – bigger than model-seed SD (0.0022) and ~4.5x the ensemble’s +0.0014 margin. Our split (0.7747) sits *below* the mean, so it is not a draw that flatters the baseline. The transformer’s 0.0805 deficit is ~13 split-SDs, so that finding is robust to the split too.

Summary scorecard

Stage	Model	Train split	Test split	Test metric	Score
2A Presence	AND-ensemble (TF-IDF + DistilBERT)	408 contracts (80%)	102 contracts (20%)	micro-F1	0.776
2A Presence	AND-ensemble	408 contracts (80%)	102 contracts (20%)	Accuracy	85.49%
2A Presence	AND-ensemble	408 contracts (80%)	102 contracts (20%)	Precision	77.18%
2A Presence	AND-ensemble	408 contracts (80%)	102 contracts (20%)	Recall	78.04%
2B Span	DistilBERT- QA	408 contracts (80%)	102 contracts (20%)	token-F1	0.763
2B Span	DistilBERT- QA	408 contracts (80%)	102 contracts (20%)	Exact Match	35.5%

Stage	Model	Train split	Test split	Test metric	Score
2B Span	DistilBERT-QA	408 contracts (80%)	102 contracts (20%)	token-F1 (window from 2A)	0.386
1 Summarizer	FLAN-T5-small	408 contracts (80%)	102 contracts (20%)	ROUGE-L (41 templates)	0.775
1 Summarizer	FLAN-T5-small	408 contracts (80%)	102 contracts (20%)	ROUGE-L (clause-specific)	0.116
Full pipeline	all three stages	408 contracts (80%)	102 contracts (20%)	gold clauses surviving	21.7%

Key design decisions

- **80/20 contract-level split** — whole contracts assigned to train or test; no clause leaks across the boundary; seeded and reused across all three models.
- **Strong baseline first** — TF-IDF on the whole document (no length limit) sets the bar; every transformer result is judged against it.
- **Measure the ceiling before training** — retrieval hit-rate (64%) told us a windowed transformer’s recall was capped below the baseline, so we pivoted to max-pool MIL.
- **Parameter-free ensemble** (AND/OR/AVG rules) — nothing tuned on the test set.
- **Quantify small margins instead of hedging about them** — a bootstrap CI turned “the margin is small so we do not lean on it” into a measured statement that it is a tie.
- **Leakage discipline** — gold offsets label training windows but are never used to build inputs or pick inference windows; retrieval uses only the category/question.
- **Honest dual eval for the summarizer** — ROUGE-L and a qualitative comparison, because ROUGE alone is misleading with only 41 targets.

Engineering notes

- **Apple MPS, no CUDA.** torch 2.8, transformers 5.x. DistilBERT + FLAN-T5-small throughout (DeBERTa-v3 unstable on MPS).
- **MPS memory does not release in-process.** Summarizer trained on CPU with `use_cpu=True` to avoid OOM.
- **transformers 5.x API:** `processing_class=...` (not `tokenizer=...`); logits cast to `fp32`; `fp16/bf16=False`, `dataloader_pin_memory=False`.

Files

```

clausify-research/
|--- README.md           - start here
|--- thesis/             - the B.Sc. thesis (LaTeX source + main.pdf)
|--- PROJECT_REPORT.md / .pdf - complete step-by-step report
|--- REPORT.md / .pdf    - this report
|--- METHODOLOGY.md / .pdf - design decisions document
|--- analysis/           - scripts generating the reported statistics
|--- data/
|   |--- train_80.csv     - 80% training split (16,728 rows, 408 contracts)
|   `--- test_20.csv      - 20% test split (4,182 rows, 102 contracts)

```

```
`--- notebooks/
|--- README.md          - cell-by-cell notebook guide
|--- train.ipynb        - training half
|--- test.ipynb         - testing half
|--- eda.ipynb          - original combined notebook
|--- artifacts/         - split + fitted TF-IDF baseline
`--- outputs/           - trained presence, span and summarizer models
```